

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.serviceloader-`
Release 1.0.1 SNAPSHOT

September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
133	Service Loader Mediator Specification	7
133.1	Introduction	7
133.2	Java Service Loader API	9
133.3	Consumers	10
133.4	Service Provider Bundles	12
133.5	Service Loader Mediator	14
133.6	osgi.serviceloader Namespace	17
133.7	Use of the osgi.extender Namespace	17
133.8	Security	18
133.9	org.osgi.service.serviceloader	18
133.10	References	19

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

133 Service Loader Mediator Specification

Version 1.0

133.1 Introduction

Java SE 6 introduced the *Service Loader*, a simple service-provider loading facility, that attempted to unify the different ad-hoc mechanisms used by Java's many factories and builders. The design allows a JAR to advertise the name of one or more embedded classes that implement a given interface and consumers to obtain instances of these implementation classes through the Service Loader API.

Though the Service Loader is about extensibility, its own design is closed and therefore not extendable. It does not support a provider model that would allow different ways of finding interface implementations; its classes are final and its policy is fixed. Unfortunately, the Service Loader's fixed design uses a non-modular class loading policy; it defines its visibility scope with a class loader, which in general requires full visibility of the application's class path. The Service Loader can therefore in OSGi not find implementations from other bundles. Additionally, the Service Loader also does not enforce a life cycle; objects are handed out forever.

Since the Service Loader is the only standardized plugin mechanism in the JRE it is necessary that the mechanism is supported in OSGi with as few changes as possible from the consumer's authors. This specification therefore defines a *mediator* that ensures that the Service Loader is useful in an OSGi Framework, allowing programs that leverage the Service Loader to be used in OSGi frameworks almost as-is.

133.1.1 Essentials

- *Compatibility* - Allow JARs that run in a classic Java SE environment that leverage the Service Loader to run in OSGi with only manifest modifications.
- *Services* - Register services for Service Provider bundles that opt-in.
- *Security* - Enforce service permissions for the Service Loader objects.
- *Life Cycle* - Manage the life cycle mismatch between OSGi bundles and the Service Loader's create only model.

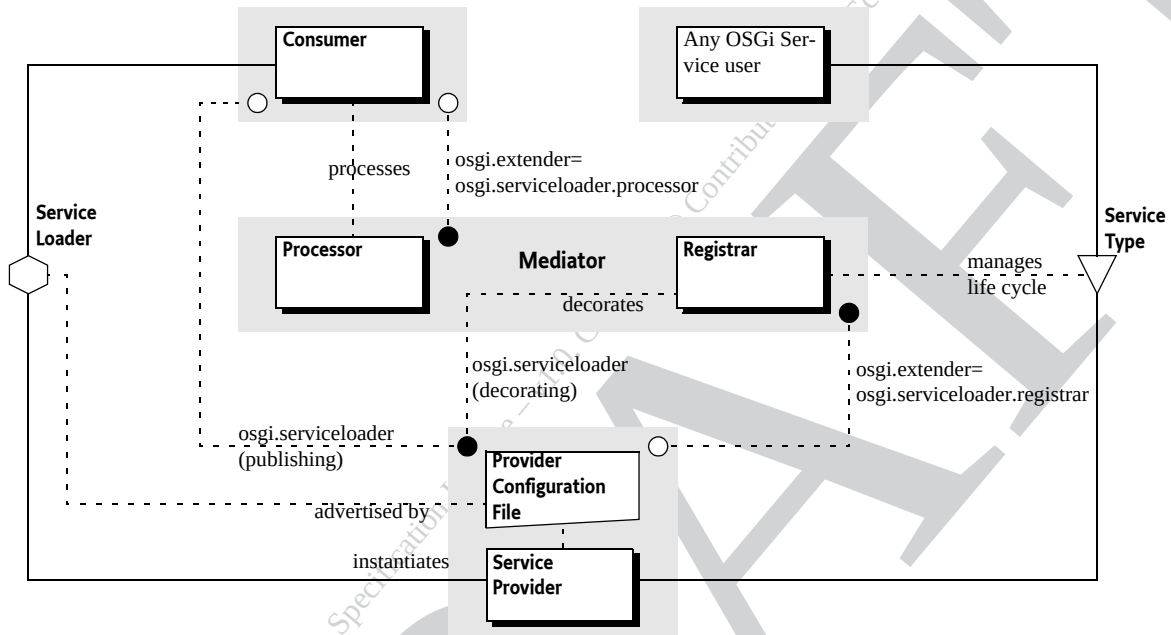
133.1.2 Entities

- *Service Loader* - An API in Java SE that allows a Consumer to find an implementation of a Service Type from a Service Provider by searching a class loader for Service Providers.
- *Service Type* - The interface or class that the Service Provider must implement/extend.
- *Provider Configuration File* - A resource in the META-INF/services directory that has the fully qualified name of the Service Type and contains one or more fully qualified names of Service Providers.
- *Service Provider* - An implementation class that implements or extends the Service Type.
- *Consumer* - A class that uses the Java SE Service Loader inside an OSGi framework.
- *Mediator* - An extender that mediates between Consumer bundles, the Service Loader API, and Service Provider bundles in an OSGi environment. It consists of a Processor and a Registrar.

- *Processor* - Modifies a bundle that uses the Service Loader API so that it works in an OSGi environment.
- *Registrar* - Registers services on behalf of a bundle that contains Service Providers.

Figure 133.1

Entities



133.1.3

Synopsis

This specification defines two different functions that are provided by a Mediator extender:

- Register OSGi services for each Service Provider.
- Allow Consumers that uses the Service Loader API to access Service Providers from other bundles that would normally not be visible from a bundle.

A Service Provider bundle can provide access to all its Service Providers through OSGi services by declaring a requirement on the `osgi.service.loader.registrar` extender. This requirement activates a Mediator to inspect the `osgi.service.loader` capabilities. If no `register` directive is used then all Service Providers for the given Service Type must be registered. Otherwise, each capability can select one Service Provider with the `register` directive. The fully qualified name selects a specific Service Provider, allowing different Service Providers to be registered with different service properties. The Mediator will then register an OSGi service factory for each selected capability. The `osgi.service-loader` capability's attributes are used to decorate the OSGi service registration with service properties. The service factory returns a new instance for each service get.

Consumers are classes that use the Service Loader API to find Service Provider instances. Since the Service Loader API requires full visibility the Service API fails to work inside an OSGi bundle. A `osgi.service.loader.processor` extender, which is the Mediator, processes bundles that require this capability by modifying calls to the Service Loader API to ensure that the Service Loader has visibility to published Service Providers.

A Consumer's bundle by default receives visibility to all published Service Providers. Service Providers are published when a bundle declares one or more `osgi.service.loader` capabilities for a Service Type. If the Consumer has an `osgi.service.loader` requirement for the given Service Type then the Mediator must only expose the bundles that are wired to those requirements and for each bundle provide all its Service Providers.

133.2 Java Service Loader API

Java is quite unique with its focus on separation of *specification* and *implementation*. Virtually all Java Specification Requests (JSR) provide a specification that can be implemented independently by different parties. Though this is one of the industry's best practices it raises a new problem: how to find the implementation available in a Java environment from only the *Service Type*. A Service Type is usually an interface but a base class can also be used.

Finding a Service Provider (the implementation class) from a Service Type is the so called *instance coupling* problem. The use of Service Types removed the type coupling between the Consumer of the contract and the *Service Provider* of the contract (the implementation) but to make things work there is a need of at least one place where the Service Provider is instantiated. The very large number of factories in Java reflects that this is a very common problem.

The general pattern for factories to find Service Providers was to search the class loaders for classes with constant names, varying the package names, often using System properties to extend the different areas to be sought. Though a general pattern based on class loading tricks emerged in the Java VM and application programs, all these patterns differed in details and places where they looked. This became harder and harder to maintain and often caused unexpected instances to be found.

The `java.util.ServiceLoader` class was therefore first introduced in Java SE 6 to provide a generic solution to this problem, see [1] *Java Service Loader API*. With this API Service Providers of a specification can now *advertise* their availability by creating a *Provider Configuration File* in their JAR in the `META-INF/services` directory. The name of this resource is the fully qualified name of the Service Type, the Service Provider provides when instantiated.

The Provider Configuration File contains a number of lines with comments or a class name that implements/extends the Service Type. For example:

```
org.example.Foo
```

A Service Provider must then advertise itself like:

```
META-INF/services/org.example.Foo:
# Foo implementation
org.acme.impl.FooImplementation
```

The Service Loader API finds all advertisers by constructing the name of the Provider Configuration File from the Service Type and then calling the `getResources` method on the provided class loader. This returns an enumeration of URLs to the advertisements. It then parses the contents of the resources; that will provide it with a list of Service Providers for the sought Service Type without duplicates. The API will return an iterator that will instantiate an object for the next available Service Provider.

To find the Configuration files for a given Service Type, the Service Loader uses a class loader. The Consumer can select the following class loaders:

- A given class loader as an argument in the call to the constructor
- The Thread Context Class Loader (TCCL)
- The system loader (when null is passed or no TCCL is set)

The class loader restricts the visibility of the Service Loader to only the resources to which the class loader has visibility. If the Service Loader has no access to the advertisement of a Service Provider then it cannot detect it and it will thus not be found.

The Service Provider is loaded from the given class loader, however, the `Class.forName` method is used, which stores it in the cache of the initiating class loader. This means that Service Providers are

not garbage collected as long as there is a resolved bundle that used the Service Loader to get that Service Provider.

In the Service Loader API, the class does not have to originate from the same JAR file as the advertisement. In OSGi this is more restricted, the advertisement must come from the same bundle or must be explicitly imported.

For example, to load a Foo instance the following code could be used:

```
ServiceLoader<Foo> sl =  
    ServiceLoader.load( Foo.class );  
Iterator<Foo> it = sl.iterator();  
if ( it.hasNext() ) {  
    Foo foo = it.next();  
    ...  
}
```

Though the Service Loader API is about extensibility and contract based programming it is in itself not extendable nor replaceable. The `ServiceLoader` class is final, it comes from a sealed JAR, and is in a java package. It also does not provide an API to provide alternate means to find implementations for a Service Type.

133.3 Consumers

Consumers are classes that are not OSGi aware and directly use the *Service Loader API*. The Service Loader has a non-modular design and Consumers therefore run into many issues when running in an OSGi framework. Consumers should therefore in general be converted to use the OSGi service layer since this solves the visibility issues, life cycle impedance mismatch, and other problems. The Consumer part of this specification is therefore a last resort to use when existing code uses the Service Loader API and cannot be modified to leverage the OSGi service layer.

133.3.1 Processing

The Service Loader Mediator can *process* the Consumer by modifying calls to the Service Loader API. This specification does not detail how the Mediator ensures that the Consumer has visibility to other Service Providers. However, a Mediator could for example set an appropriate Thread Context Class Loader during the call to the Service Loader's constructor by weaving the Consumer's byte codes.

133.3.2 Opting In

Processing is an opt-in process, the Consumer bundle must declare that it is willing to be processed. The opt-in is handled by a requirement to the `osgi.serviceloader.processor` extender. This requirement must have a single cardinality (the default) since the Mediator uses the wiring to select the Consumer to process when multiple Mediators are present.

For example, the following requirement in a manifest enables a bundle to be processed:

```
Require-Capability:  
    osgi.extender;  
        filter:="(&(osgi.extender=osgi.serviceloader.processor)  
                (version>=1.0) (! (version>=2.0)))"
```

If the extender `osgi.serviceloader.processor` requirement is satisfied then the wired Mediator must process the Consumer.

The Mediator must give visibility to all bundles with *published* Service Providers unless the Consumer restricts the visibility by having `osgi.serviceloader` requirements. Bundles publish a Service

Type, meaning all their Service Providers for that type, by having at least one `osgi.serviceloader` capability for that Service Type.

133.3.3

Restricting Visibility

A Consumer's bundle can restrict its visibility to certain bundles by declaring an `osgi.serviceloader` requirement for each Service Type it wants to use. Only bundles wired from those requirement provide their advertised Service Providers. If no such requirements are declared then all bundles with the published Service Type become available.

The cardinality can be used to select a single Service Provider's bundle or multiple bundles if it needs to see all Service Provider bundles. The requirement can be made optional if the Consumer's bundle can work also when no Service Provider bundle is available. See *osgi.serviceloader Namespace* on page 17 for more details.

For example, a requirement that restricts visibility to the `org.example.Foo` Service Providers could look like:

Require-Capability:

```
osgi.serviceloader;  
filter:="(osgi.serviceloader=org.example.Foo)";  
cardinality:=multiple
```

In this example, any bundle that publishes the `org.example.Foo` Service Type will contribute its Service Providers.

Visibility can also be restricted to bundles that publish with capability's attributes. Any bundle that has at least one matching capability will then be able to contribute all its Service Providers. For example, the following example selects only bundles that have the `classified` property set:

```
osgi.serviceloader; filter:="(classified=*)"
```

With Service Registrations, see *Registering Services* on page 14, the capability can discriminate between multiple Service Providers in the same bundle. The Service Loader API does not have this feature: any wired requirement has visibility to all Service Providers in the wired bundle, regardless of the `register` directive.

133.3.4

Life Cycle Impedance Mismatch

A Consumer can only see Service Provider instances of bundles that are active during the time the next instance is created. That is, the Mediator must treat the life cycle of the Service Provider as if it was a service. However, the Service Loader implementations perform extensive class loader techniques and cache results. The exact life cycle of the Service Provider bundle with respect to the Consumer is therefore impossible to enforce.

The Service Loader API does not have a life cycle, objects are assumed to stay alive during the duration of the VM's process and due to the use of `Class.forName` in the Service Loader implementations. Therefore a Mediator should refresh a Consumer bundle when it is using a Service Provider and that Service Provider's bundle becomes stopped otherwise long running applications can run out of memory when bundles are regularly updated.

133.3.5

Consumer Example

A legacy JAR for which there is no more source code uses the Service Loader API to get access to `com.example.Codec` instances through the Service Loader API.

It is wrapped in a bundle that then has the following manifest:

```
Manifest-Version:      1.0  
Bundle-ManifestVersion: 2
```

```

Bundle-SymbolicName:    com.example.impl
Bundle-Version:         23.98.1.v199101232310.02011
Import-Package:         com.example; version=3.45
Bundle-ClassPath:       legacy.jar

```

The manifest must then declare that the bundle must be processed, this is triggered by requiring the `osgi.serviceloader.processor` extender:

```

Require-Capability:
  osgi.extender;
    filter:="(&(osgi.extender=osgi.serviceloader.processor)
              (version>=1.0) (! (version>=2.0)))"

```

With this manifest, the Consumer bundle has full visibility to all Service Provider bundles that are published. The following lines can be added to restrict the visibility to codecs that have support for WAVE formats (although all Service Providers in that bundle will be visible to the consumer).

```

,
  osgi.serviceloader;
    filter:="(&(format=WAVE) (osgi.serviceloader=com.example.Codec))"

```

133.4 Service Provider Bundles

A *Service Provider bundle* is a bundle that contains one or more Service Providers that are usable by the Service Loader API. This section shows how Service Provider bundles should be constructed and what options they have.

133.4.1 Advertising

Service Providers are implementation classes that are *advertised* under a Service Type according to the rules in the Service Loader API. A Service Provider is advertised with a *Provider Configuration File* in a JAR. In an OSGi environment the Service Provider must reside in the same bundle as the advertisement or be imported. A single Provider Configuration File can contain multiple Service Providers. See *Java Service Loader API* on page 9 .

133.4.2 Publishing the Service Providers

Service Providers can be used in two different scenarios:

- A Service Provider can be used by a processed Consumer as a Service Type, or
- It can be registered as a service.

A Service Type must be *published* to allow its use in these scenarios. Publishing a Service Type consists of providing one or more `osgi.serviceloader` capabilities for an advertised Service Type, see *osgi.serviceloader Namespace* on page 17 . These `osgi.serviceloader` capabilities must specify a fully qualified class name of the Service Type, there is no wildcarding allowed. Therefore, publishing a service implicitly makes all corresponding Service Providers available to Consumers.

If a bundle does not provide `osgi.serviceloader` capabilities then it does not publish any Service Providers and its Service Providers can therefore not be used by Consumers. They can then also not be registered as OSGi services, see *OSGi Services* on page 13 . Tools can use the advertisement of the Service Provider in the JAR to automatically generate the `osgi.serviceloader` capabilities in the manifest.

For example, the following capability publishes all the Service Providers in its bundle that advertise the `com.example.Codec` interface:

```

Provide-Capability:
  osgi.serviceloader;
    osgi.serviceloader=com.example.Codec;
    uses:="com.example"

```

A Service Provider bundle must not require the `osgi.serviceloader.processor` extender unless it needs to be processed; publishing a Service Type is sufficient to allow Consumers to use the published Service Types.

133.4.3

OSGi Services

The Service Provider can have its `osgi.serviceloader` capabilities be registered as services that provide instances from the Service Providers. For this, the Service Provider bundle must require the `osgi.serviceloader.registrar` extender, which is the Mediator. For example:

```

Require-Capability:
  osgi.extender;
    filter:="(&(osgi.extender=osgi.serviceloader.registrar)
      (version>=1.0) (! (version>=2.0)))"

```

The registrar must then inspect each `osgi.serviceloader` capability and register an associated OSGi Service for each Service Provider *selected* by that capability. A Service Provider is selected when:

- The capability has no `register` directive, or
- The `register` directive matches the fully qualified name of the Service Provider.

A `register` directive selects a Service Provider if it contains the fully qualified name of the Service Provider, that is, the implementation class. Selection only works for services, Consumer will always see all Service Providers regardless of the `register` directive due to limitations in the Service Loader API.

For example, the following manifest selects all Service Providers of the `com.example.Foo` Service Type since no `register` directive is present:

```

Provide-Capability:
  osgi.serviceloader;
    uses:="com.example";
    osgi.serviceloader=com.example.Foo

```

Selected Service Providers must be registered as defined in *Registering Services* on page 14 , with the capability's attributes as *decorating* service properties. Private service properties (attributes that start with a full stop (`'\u002E'`) and the defined capability attributes in the `osgi.serviceloader` namespace are not registered as service properties.

The following example would register the `format` service property but not the `.hint` service property for the `com.acme.impl.WaveFoo` Service Provider.

```

osgi.serviceloader;
  osgi.serviceloader=com.example.Foo;
  uses:="com.example";
  format=WAVE;
  .hint=E5437Qy7;
  register:="com.acme.impl.WaveFoo"

```

The Mediator must only register OSGi services for selected Service Providers; the Service Provider bundle can therefore decide not to register certain Service Providers and register them with another mechanism, for example Declarative Services or in a bundle activator.

Since the Mediator must use the bundle context of the Service Provider to register the OSGi service the Service Provider bundle must have the proper Service Permission REGISTER for the Service Type.

133.4.4 Service Provider Example

A Foo Codecs JAR needs to be ported to OSGi, it provides a Service Provider for the org.example.Codec Service Type. In this example the JAR is given a new manifest:

```
Manifest-Version:      1.0
Bundle-ManifestVersion: 2
Bundle-SymbolicName:   com.example.foo.codecs
Import-Package:        com.example; version=3.45
```

To ensure that the bundle opts in to registering its services it must require the osgi.service-loader.registrar extender.

```
Require-Capability:
  osgi.extender;
    filter:="(&(osgi.extender=osgi.serviceloader.registrar)
              (version>=1.0) (! (version>=2.0)))"
```

To publish two Service Providers for the same type, two capabilities must be declared:

```
Provide-Capability:
  osgi.serviceloader;
    osgi.serviceloader="com.example.Codec";
    format:List<String>="WAVE,WMF";
    register:="com.acme.impl.FooWaveCodec";
    uses:="com.example,org.apache.common.codecs",
  osgi.serviceloader;
    osgi.serviceloader="com.example.Codec";
    format:List<String>="SINUS";
    register:="com.acme.impl.sinus.FooSinusCodec";
    uses:="com.example"
```

This example implicitly publishes the Service Type com.example.Codec multiple times with different attributes. Consumers that match any of these capabilities will however have visibility to all Service Providers since the Service Loader API cannot discriminate between different Service Providers from the same bundle.

133.5 Service Loader Mediator

A Mediator is the osgi.serviceloader.processor and osgi.serviceloader.registrar extender bundle that has the following responsibilities:

- It registers selected Service Providers as OSGi services.
- It processes any Consumers so that Service Loader API calls have proper visibility to published Service Provider bundles.

133.5.1 Registering Services

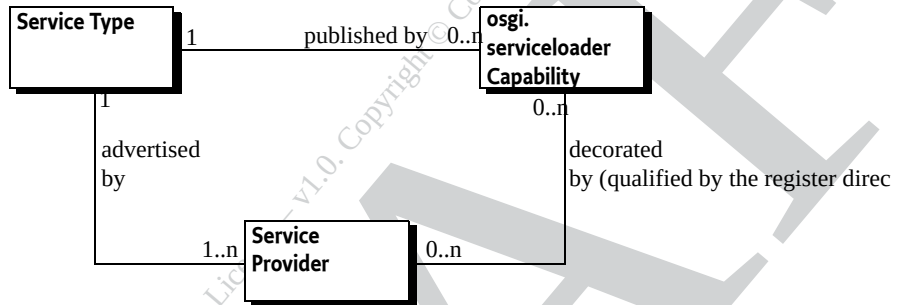
The Mediator must track bundles that are wired to its osgi.extender=osgi.serviceloader.registrar capability. These are called the *managed* bundles. For all managed bundles the Mediator must enumerate all osgi.serviceloader capabilities and register *selected* Service Providers as OSGi services. A Service Provider is selected by an osgi.serviceloader capability when:

- The advertised Service Type matches the corresponding `osgi.serviceloader` capability's Service Type, and
- The register directive is absent, or
 - The register directive contains the fully qualified name of the Service Provider.

An `osgi.serviceloader` capability that selects a Service Provider is said to *decorate* that Service Provider. A capability can decorate multiple Service Providers of the same Service Type and the same Service Provider can be decorated by different capabilities. Figure 133.2 depicts the resulting relations and their cardinalities since the relations are non-trivial.

Figure 133.2

Cardinality Service Type



The OSGi service for each selected Service Provider must be registered under the advertised Service Type of the Service Provider, which must match the Service Type specified in the capability.

133.5.2

OSGi Service Factory

The Mediator must register an OSGi service factory with the bundle context of the Service Provider's bundle. The OSGi service factory must be implemented such that it creates a new instance for each bundle that gets the service. This behavior is similar, though not quite identical, to the `ServiceLoader.load()` method that gives each consumer a separate instance of the service. The difference is that different users inside a bundle will share the same instance.

Each service registration is controlled by a decorating `osgi.serviceloader` capability. The attributes on this capability must be registered with the OSGi service as service properties, except for:

- *Private* - Private properties, property names that start with a full stop (' \u002E) must not be registered.

The following service property must be registered, overriding any identical named properties in the decorating capability:

- `serviceloader.mediator` - (Long) The bundle id of the mediator.

The Mediator should not verify class space consistency since the OSGi framework already enforces this as long as the publishing capability specifies the `uses` directive.

Any services registered in the OSGi Service Registry must be unregistered when the Service Provider's bundle is stopped or the Mediator is stopped.

133.5.3

Service Loader and Modularity

The Service Loader API causes issues in a modular environment because it requires a class loader that has wide visibility. In a modular environment like OSGi the Consumer, the Service Type, and the Service Provider can, and should, all reside in different modules because they represent different concerns. Best practice requires that only the Service Type is shared between these actors. However,

for the Service Loader to work as it was designed the Consumer must provide a class loader that has visibility of the Service Provider. The Service Provider is an implementation class, exporting such classes is the anathema of modularity. However, since the standard JRE provides application wide visibility this was never a major concern.

The simplest solution is to make the Service Loader aware of OSGi, its API clear is mappable to the OSGi service layer. However, the Service Loader is not extensible. The result is that using the Service Loader in OSGi fails in general because the Service Loader is unable to find the Service Providers. The issues are:

- The use of the Thread Context Class Loader (TCCL) is not defined in an OSGi environment. It should be set by the caller and this cannot be enforced. The multi threaded nature of OSGi makes it hard to predict what thread a Consumer will use, making it impossible to set an appropriate TCCL outside the Consumer.
- A bundle cannot import META-INF/services since the name is not a package name. Even if it could, the OSGi framework can only bind a single exporter to an importer for a given package. The Service Loader API requires access to all these pseudo-packages via the Class Loader's `getResources` method, the technique used to find Service Providers.
- Instantiating a Service Provider requires access to internal implementation classes, by exporting these classes, an implementing bundle would break its encapsulation.
- If a Service Provider was exported then importing this class in a Consumer bundle would couple it to a specific implementation package; this also violates the principle of loose coupling.
- The Service Loader API does assume an eternal life cycle, there is no way to signal that a Service Provider is no longer available. This is at odds with the dynamic bundle life cycle.

133.5.4 Processing Consumers

Consumers are not written for OSGi and require help to successfully use the Service Loader API. It is the Mediator's responsibility to ensure that bundles that are wired to published Service Types have access to these Service Provider's instances through the Service Loader API.

This specification does not define how this is done. There are a number of possibilities and it is up to the Mediator to provide the guarantee to the Consumer that it has been properly processed.

A Mediator must only process Consumer's bundles that are wired to the `osgi.extender` capability for the `osgi.serviceloader.processor` extender. Since Consumers must require this extender capability with the default cardinality of 1 there can at most be one extender wired to a Consumer.

133.5.5 Visibility

The Mediator must process the Consumer bundle in such a way that when the Consumer uses the Service Loader API it receives all the Service Providers of bundles that:

- Provide one or more `osgi.serviceloader` capabilities for the requested Service Type, and
- Are not type space incompatible with the requester for the given Service Type, and
- Either the Consumer has no `osgi.serviceloader` requirements or one of its requirements is wired to one of the `osgi.serviceloader` capabilities.

The Mediator must verify that the Consumer has Service Permission GET for the given Service Type since the Consumer uses the Service Type as a service. This specification therefore reuses the Service Permission for this purpose. The check must be done with the `ServicePermission(String,String)` constructor using the bundle's Access Control Context or the bundle's `hasPermission` method.

133.5.6 Life Cycle

There is a life cycle mismatch between the Service Loader API and the dynamic OSGi world. A Service Loader provides a Consumer with an object that could come from a bundle that is later stopped

and/or refreshed. Such an object becomes *stale*. Mediators should attempt to refresh bundles that have access to these stale objects.

133.6

osgi.serviceloader Namespace

The `osgi.serviceloader` Namespace:

- Allows the Consumer's bundle to require the presence of a Service Provider for the required Service Type.
- Provides the service properties for the service registration.
- Indicates which Service Providers should be registered as an OSGi service.

The namespace is defined in the following table and [ServiceLoaderNamespace](#), see ??? for the legend of this table.

Table 133.1 *osgi.serviceloader namespace definition*

Name	Kind	M/O	Type	Syntax	Description
osgi.serviceloader	CA	M	String	qname	The Service Type's fully qualified name.
*	CA	O	*	*	Additional matching attributes are permitted. These attributes will be registered as custom service properties unless they are private (start with a full stop).
register	CD	O	String	qname	Use this capability to register a different Service Factory under the Service Type for each selected Service Provider. A Service Provider is selected if the Service Type is the advertising Service Type and the Service Provider's fully qualified name matches the given name. If no register directive is present all advertised Service Providers must be registered. To register no Service Providers, because the capability must only be used to publish, provide an empty string.

133.7

Use of the osgi.extender Namespace

This section specifies the extender names for Mediators. They are used by both by Consumer and Service Provider bundles to ensure that a Mediator is present. Both names are defined for the general `osgi.extender` namespace in *osgi.extender Namespace* in ???.

The `osgi.extender` namespace requires the use of an *extender name*, the name of the Mediator extends is:

```
osgi.serviceloader.processor
osgi.serviceloader.registrar
```

The version is for this specification is in both cases:

1.0.0

133.8 Security

133.8.1 Mediator

The Mediator will require significant permissions to perform its tasks. First, it will require access to the Bundle Context of the Service Provider bundle, which means it must have Admin Permission:

```
AdminPermission[<Service Provider Bundles>,CONTEXT|METADATA|CLASS]
```

Since it will have to register on behalf of the Service Provider bundle it must have complete liberty to register services:

```
ServicePermission[<Service Type>,REGISTER]
```

Depending on the way the Consumers are processed additional requirements may be necessary.

The Mediator connects two parties; it must ensure that neither party will receive additional permissions.

133.8.2 Consumers

Consumers must have:

```
ServicePermission[<Service Type>,GET]
PackagePermission[<Service Type's package>,IMPORT]
CapabilityPermission["osgi.extender", REQUIRE]
CapabilityPermission["osgi.serviceloader", REQUIRE]
```

The Mediator must ensure that the Consumer has the ServicePermission before it provides the instance. It must use the Bundle Context hasPermission method or the bundle's Access Control Context to verify this.

133.8.3 Service Providers

Service Providers must have:

```
ServicePermission[<Service Type>,REGISTER]
PackagePermission[<Service Type's package>,IMPORT]
CapabilityPermission["osgi.extender", REQUIRE]
CapabilityPermission["osgi.serviceloader", PROVIDE]
```

The Mediator must ensure that the Service Provider has the ServicePermission before it provides the instance. It must use the Bundle Context hasPermission method or the bundle's Access Control Context to verify this.

133.9 org.osgi.service.serviceloader

Service Loader Mediator Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.serviceloader; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.serviceloader; version="[1.0,1.1]"}
```

133.9.1

Summary

- `ServiceLoaderNamespace` - Service Loader Capability and Requirement Namespace.

133.9.2

public final class ServiceLoaderNamespace extends Namespace

Service Loader Capability and Requirement Namespace.

This class defines the names for the attributes and directives for this namespace.

All unspecified capability attributes are of one of the following types:

- `{@code String}`
- `{@code Version}`
- `{@code Long}`
- `{@code Double}`
- `{@code List<String>}`
- `{@code List<Version>}`
- `{@code List<Long>}`
- `{@code List<Double>}`

and are used as arbitrary matching attributes for the capability. The values associated with the specified directive and attribute keys are of type `{@code String}`, unless otherwise indicated.

All unspecified capability attributes, unless the attribute name starts with full stop (`{@code '.'}` \u002E), are also used as service properties when registering a Service Provider as a service.

Concurrency Immutable

133.9.2.1

public static final String CAPABILITY_REGISTER_DIRECTIVE = "register"

The capability directive used to specify the implementation class of the service. The value of this directive must be of type `{@code String}`.

If this directive is not specified, then all advertised Service Providers that match the service type name must be registered. If this directive is specified, then only Service Providers that match the service type name whose implementation class matches the value of this attribute must be registered. To not register a service for this capability use an empty string.

133.9.2.2

public static final String SERVICELOADER_NAMESPACE = "osgi.serviceloader"

Namespace name for service loader capabilities and requirements.

Also, the capability attribute used to specify the fully qualified name of the service type.

133.10 References

- [1] *Java Service Loader API*
<https://docs.oracle.com/javase/8/docs/api/java/util/ServiceLoader.html>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT